

파일 1. AGENTS.md

Project Instructions for Codex

Project Goal

Build a domain-agnostic ****Visual SOP Graph Studio POC****.

This project is not a fixed SOP management screen and not a liquid-hydrogen-only application.

It is a visual programming editor where users freely compose executable SOP logic by connecting nodes with typed input/output ports.

The interface should feel closer to Unreal Blueprint, Node-RED, Rete.js-style graph editing, or a mind-map-based node editor than a traditional CRUD admin screen.

Core Product Concept

Users create an executable ``SOPGraph`` by dragging node types into an infinite canvas and connecting them through typed ports.

A graph may combine:

- Event nodes
- Space / Scope nodes
- Asset / Object nodes
- Condition / Branch nodes
- SOP Task nodes
- Composite SOP Group nodes
- Role / Contact nodes
- Notification nodes
- Timer / Escalation nodes
- Situation Board nodes
- Evidence / Record nodes

The generated graph is compiled into an executable SOP flow.

MVP Scope

Implement the first vertical slice:

1. Main Graph Studio screen
2. Left Node Palette
3. Infinite React Flow canvas
4. Custom nodes with visible typed input/output ports
5. Right Property Inspector
6. Composite SOP Group node with expand/collapse
7. Graph validation panel
8. Compile current nodes/edges into `SOPGraph` JSON
9. Simulate an `EventContext`
10. Highlight the execution path
11. Runtime Preview showing generated missions and mock status results

Required Sample Flow

The first POC must support this sample graph:

```
```text
```

Event

- Space Scope
- Asset Filter
- Condition
- SOP Group
- Notification
- Branch
- Situation Board

Example:

가스누출 감지

- 발생 공간 범위 적용
- 관련 객체 필터
- 위험도 WARNING 이상 조건
- 가스누출 초동대응 SOP Group
- SMS/앱 전파 Mock
- 응답 분기
- 전자상황판 기록

## Non-goals

Do not implement the following in the first POC:

- Real SMS
- Real IoT / sensor integration
- Real 3D Viewer
- Production authentication
- Production database
- AI/RAG SOP generation
- Domain-specific KGS-only screens

Use liquid hydrogen, power plant, and disaster drill only as sample templates.

---

## Technology Preference

Use:

- React
- TypeScript
- Vite
- React Flow / xyflow

Recommended package:

```
npm install @xyflow/react
```

---

## Architecture Rule

React Flow is only the visual editor.

The product data model must be `SOPGraph`, not React Flow state itself.

```
React Flow Nodes / Edges
 ↓
normalizeGraph()
 ↓
SOPGraph
 ↓
validateGraph()
 ↓
compileGraph()
 ↓
ExecutionPlan
```

↓  
Runtime Preview

Keep the editor layer and execution logic separated.

## Acceptance Criteria

The POC is successful when:

1. A user can add nodes from the Node Palette.
2. Nodes have typed input/output ports.
3. A user can connect nodes through visible handles.
4. Invalid or missing connections are detected.
5. A user can create the sample flow:  
Event → Space Scope → Asset Filter → Condition → SOP Group → Notification → Branch → Situation Board
6. A composite SOP Group can be expanded/collapsed.
7. The graph can be compiled to SOPGraph JSON.
8. A simulated EventContext can trigger a graph execution preview.
9. The execution path is highlighted.
10. Runtime Preview shows mock missions, notification status, and situation board records.

```

파일 2. `docs/00_PRD_POC.md`

```md
# 00_PRD_POC.md
# Visual SOP Graph Studio 1차 POC PRD

## 1. 제품 개요

Visual SOP Graph Studio는 공간, 객체, 이벤트, 조건, SOP 임무, 담당자, 전파, 상황
판을 노드로 표현하고, 사용자가 이를 자유 캔버스에서 연결하여 실행 가능한 `SOPGraph`
를 생성하는 시각적 SOP 프로그래밍 도구이다.

본 POC는 액화수소 도메인 전용 시스템이 아니다.
장기적으로 플랜트, 발전소, 공공시설, 학교, 지하공간, 산업단지, 안전한국훈련 등 다양한
재난안전 상황에 적용 가능한 범용 SOP 그래프 저작 도구를 목표로 한다.

---

## 2. 기존 방향과의 차이

기존 1차 POC는 다음 구조였다.

```text
```

Space / Asset 조회  
→ Event 선택  
→ SOPBinding 생성  
→ Event Simulator  
→ Situation Board

수정된 방향은 다음 구조이다.

Node Palette  
→ Infinite Canvas  
→ Typed Port Connection  
→ SOPGraph Validation  
→ SOPGraph Compile  
→ EventContext Simulation  
→ Runtime Preview

기존 구조가 “SOP 매핑 관리 화면”에 가까웠다면, 수정된 구조는 “SOP를 시각적으로 프로그래밍하는 저작 도구”에 가깝다.

### 3. 제품 정의

본 시스템의 핵심은 사용자가 다음 요소를 노드로 자유롭게 배치하고 연결할 수 있게 하는 것이다.

Event  
Space  
Asset  
Condition  
SOP Task  
SOP Group  
Role  
Notification  
Branch  
Timer  
Escalation  
Situation Board  
Evidence

노드는 각각 속성, 입력 포트, 출력 포트를 가진다.

노드 간 연결은 단순 선이 아니라 실행 흐름, 데이터 흐름, 조건 흐름, 전파 흐름을 의미한다.

## 4. 1차 POC 목표

1차 POC의 목표는 다음과 같다.

목표	설명
자유 캔버스	사용자가 노드를 자유롭게 배치
노드 팔레트	다양한 SOP 구성 노드를 제공
포트 연결	노드의 입력/출력 포트를 연결
속성 편집	선택 노드의 속성을 우측 패널에서 수정
복합 노드	SOP Group을 접고 펼칠 수 있음
그래프 검증	필수 연결, 필수 속성, 연결 오류 검증
그래프 컴파일	React Flow 상태를 SOPGraph JSON으로 변환
시뮬레이션	EventContext를 입력해 실행 경로 확인
실행 미리보기	Runtime Preview에서 임무/전파/응답 Mock 확인

## 5. 주요 사용자

사용자	역할
안전관리자	재난/안전 SOP 시나리오를 그래프 형태로 작성
상황실 운영자	이벤트 발생 시 실행 경로와 임무 상태 확인
시스템 관리자	노드 템플릿, 담당자, 전파 채널 관리
연구개발 담당자	도메인별 템플릿, 실행엔진, AI/RAG 확장

## 6. POC 포함 범위

1차 POC에는 다음을 포함한다.

- React Flow 기반 무한 캔버스
- Node Palette
- Custom Graph Node
- Typed Input / Output Port
- Property Inspector
- Composite SOP Group
- Collapse / Expand
- Graph Validation
- SOPGraph JSON Export
- EventContext Simulator
- Runtime Preview

## 7. POC 제외 범위

1차 POC에서는 다음을 제외한다.

- 실제 SMS 발송
- 실제 앱푸시
- 실제 센서/IoT 연동
- 실제 3D Viewer 연동
- 실제 DB 저장
- 로그인/권한
- AI/RAG 기반 SOP 자동 생성
- 상용 운영 수준의 전자상황판

## 8. 도메인 확장 방향

1차 POC에서는 액화수소를 하나의 샘플로만 사용한다.

샘플 도메인	예시 이벤트	예시 공간	예시 SOP
액화수소 플랜트	누출의심, 압력상승	저장구역, 벤트구역	초동대응, 전파, 보고
발전소	터빈 진동 이상	터빈실, 전기실	점검, 부하조정, 책임자 보고
안전한국훈련	지진, 화재, 침수	건물, 대피구역	대피유도, 인원확인, 상황보고
일반 사업장	작업자 쓰러짐, 화재감지	작업구역, 위험구역	신고, 응급조치, 대피

## 9. 성공 기준

1차 POC 성공 기준은 다음과 같다.

1. 사용자가 노드를 캔버스에 자유롭게 추가할 수 있다.
2. 노드 간 입력/출력 포트를 연결할 수 있다.
3. Event → Space Scope → Asset Filter → Condition → SOP Group → Notification → Branch → Situation Board 흐름을 만들 수 있다.
4. SOP Group을 접고 펼칠 수 있다.
5. 그래프 검증 결과를 확인할 수 있다.
6. SOPGraph JSON을 생성할 수 있다.
7. EventContext 시뮬레이션으로 실행 경로를 확인할 수 있다.
8. Runtime Preview에서 임무와 상태 결과를 확인할 수 있다.

---

# 파일 3. `docs/01\_DOMAIN\_MODEL.md`

```

```md
# 01_DOMAIN_MODEL.md
# Domain Model

## 1. 모델 전환 개요

기존 모델의 중심은 `SOPBinding`이었다.

```text
Event + Space + Asset → SOP

```

수정된 모델의 중심은 `SOPGraph`이다.

```

SOPGraph
= Nodes

+ Edges
+ Ports
+ Groups
+ Variables
+ Validation
+ ExecutionPlan

```

즉, 사용자가 만든 결과물은 단순 매핑 규칙이 아니라 실행 가능한 SOP 그래프이다.

## 2. SOPGraph

```

export interface SOPGraph {
 graphId: string;
 name: string;
 description?: string;
 domain: string;
 version: string;
 nodes: GraphNode[];
 edges: GraphEdge[];
 groups?: GraphGroup[];
 variables?: GraphVariable[];
 validation?: ValidationResult;
 createdAt: string;
 updatedAt: string;
}

```

## 필드 설명

필드	설명
graphId	SOPGraph 고유 ID
name	그래프 이름
description	설명
domain	generic_safety, plant, power_plant, disaster_drill 등
version	버전
nodes	그래프 노드 목록
edges	연결선 목록
groups	복합 노드/그룹 정보
variables	그래프 실행 변수
validation	검증 결과
createdAt	생성일시
updatedAt	수정일시

## 3. GraphNode

```
export interface GraphNode {
 id: string;
 type: NodeType;
 label: string;
 description?: string;
 properties: Record<string, unknown>;
 ports: NodePort[];
 position: {
 x: number;
 y: number;
 };
 size?: {
 width: number;
 height: number;
 };
 collapsed?: boolean;
 children?: string[];
}
```

## 4. NodeType

```
export type NodeType =
 | "event"
 | "space_scope"
 | "asset_filter"
 | "condition"
 | "branch"
 | "sop_task"
 | "sop_group"
 | "role"
 | "notification"
 | "timer"
 | "escalation"
 | "situation_board"
 | "evidence"
 | "data_mapper";
```

## 5. NodePort

```
export interface NodePort {
 id: string;
 label: string;
 direction: "input" | "output";
 dataType: PortDataType;
 required?: boolean;
 multiple?: boolean;
}
```

## 6. PortDataType

```
export type PortDataType =
 | "event"
 | "scoped_event"
 | "asset"
 | "condition_result"
 | "execution_flow"
 | "mission"
 | "mission_status"
 | "notification_payload"
```

```
| "response"
| "record";
```

## 7. GraphEdge

```
export interface GraphEdge {
 id: string;
 sourceNodeId: string;
 sourcePortId: string;
 targetNodeId: string;
 targetPortId: string;
 edgeType: EdgeType;
 label?: string;
 condition?: Record<string, unknown>;
}
```

## 8. EdgeType

```
export type EdgeType =
 | "event_flow"
 | "scope_flow"
 | "asset_flow"
 | "condition_flow"
 | "execution_flow"
 | "notification_flow"
 | "response_flow"
 | "record_flow";
```

## 9. EventContext

이벤트 발생 시 그래프 실행에 입력되는 표준 이벤트 객체이다.

```
export interface EventContext {
 eventId: string;
 eventType: string;
 severity: "INFO" | "CAUTION" | "WARNING" | "DANGER" | "CRITICAL";
 occurredAt: string;
 siteId?: string;
 spaceId?: string;
 assetId?: string;
```

```
source: "sensor" | "manual" | "ai" | "simulation";
measuredValues?: Record<string, number | string | boolean>;
metadata?: Record<string, unknown>;
}
```

---

## 10. ExecutionPlan

컴파일된 SOPGraph가 실제 실행될 때 생성되는 실행 계획이다.

```
export interface ExecutionPlan {
 planId: string;
 graphId: string;
 triggerNodeId: string;
 executionPath: string[];
 missions: RuntimeMission[];
 notifications: RuntimeNotification[];
 status: "READY" | "RUNNING" | "COMPLETED" | "FAILED";
}
```

---

## 11. RuntimeMission

```
export interface RuntimeMission {
 missionId: string;
 nodeId: string;
 title: string;
 assigneeRole?: string;
 status: "SENT" | "RUNNING" | "COMPLETED" | "DELAYED" | "FAILED";
 dueMinutes?: number;
}
```

---

## 12. ValidationIssue

```
export interface ValidationIssue {
 level: "error" | "warning" | "info";
 nodeId?: string;
 edgeId?: string;
 message: string;
}
```

---

# 파일 4. `docs/02\_USER\_FLOW.md`

```md

02_USER_FLOW.md

User Flow

1. 사용자 흐름 개요

수정된 1차 POC는 기존의 목록/상세 중심 흐름이 아니라, 자유 캔버스에서 노드를 배치하고 연결하여 SOPGraph를 생성하는 흐름을 중심으로 한다.

Flow 1. Graph Studio 진입

1. 사용자는 Visual SOP Graph Studio에 접속한다.
2. 화면 중앙에는 무한 캔버스가 표시된다.
3. 좌측에는 Node Palette가 표시된다.
4. 우측에는 Property Inspector가 표시된다.
5. 하단에는 Validation / Compile Panel이 표시된다.

Flow 2. 노드 추가

1. 사용자는 좌측 Node Palette에서 노드 유형을 선택한다.
2. 노드를 캔버스로 드래그앤드롭한다.
3. 시스템은 해당 노드 유형의 기본 속성과 포트를 가진 GraphNode를 생성한다.
4. 사용자는 노드를 자유롭게 이동하여 배치한다.

예시 노드:

```text

Sensor Event

Space Scope

Asset Filter

Condition

SOP Group

Notification

Branch

Situation Board

## Flow 3. 노드 연결

1. 사용자는 Event Node의 output port를 Space Scope Node의 input port에 연결한다.
2. Space Scope Node의 output port를 Condition Node에 연결한다.
3. Asset Filter Node를 Condition Node에 연결한다.
4. Condition Node의 true output을 SOP Group Node에 연결한다.
5. SOP Group Node를 Notification Node에 연결한다.
6. Notification Node를 Branch Node에 연결한다.
7. Branch Node를 Situation Board Node에 연결한다.

예시:

```
Event
→ Space Scope
→ Asset Filter
→ Condition
→ SOP Group
→ Notification
→ Branch
→ Situation Board
```

## Flow 4. 노드 속성 편집

1. 사용자는 캔버스에서 노드를 클릭한다.
2. 우측 Property Inspector에 선택 노드의 속성이 표시된다.
3. 사용자는 이벤트 유형, 조건식, 담당자, 제한시간, 전파채널 등을 수정한다.
4. 변경 내용은 GraphNode의 properties에 반영된다.

## Flow 5. 복합 SOP Group 작성

1. 사용자는 SOP Group Node를 추가한다.
2. SOP Group을 펼친다.
3. 내부에 SOP Task Node를 여러 개 추가한다.
4. Task 간 순서를 연결한다.
5. 각 Task에 담당 Role, 완료 조건, 제한 시간을 설정한다.
6. SOP Group을 접으면 하나의 복합 노드로 표시된다.

## Flow 6. 그래프 검증

1. 사용자는 Validate 버튼을 클릭한다.
2. 시스템은 필수 포트 연결 여부를 확인한다.
3. 필수 속성 누락 여부를 확인한다.
4. 잘못된 포트 타입 연결을 확인한다.
5. 순환 참조 또는 도달 불가능 노드를 확인한다.

6. 하단 Validation Panel에 결과를 표시한다.

---

## Flow 7. SOPGraph 컴파일

1. 사용자는 Compile 버튼을 클릭한다.
2. 시스템은 React Flow nodes/edges를 SOPGraph JSON으로 변환한다.
3. SOPGraph JSON Preview를 표시한다.
4. 컴파일 성공 시 Runtime Preview 실행이 가능해진다.

---

## Flow 8. EventContext 시뮬레이션

1. 사용자는 샘플 EventContext를 선택한다.
2. 시스템은 Event Node와 EventContext를 매칭한다.
3. Condition Node를 평가한다.
4. 실행 가능한 경로를 계산한다.
5. 캔버스에서 실행 경로를 하이라이트한다.

---

## Flow 9. Runtime Preview

1. 실행 경로를 기준으로 임무가 생성된다.
2. Notification Node에서 전파 메시지 Mock이 생성된다.
3. Branch Node에서 응답/미응답 분기가 표시된다.
4. Situation Board Node에 기록될 내용이 표시된다.
5. 사용자는 실제 실행 전 SOPGraph의 동작을 확인한다.

---

## Flow 10. 도메인 템플릿 불러오기

1. 사용자는 Template Library를 연다.
2. 액화수소 플랜트, 발전소, 안전한국훈련, 일반 사업장 안전 템플릿 중 하나를 선택한다.
3. 선택한 템플릿은 SOPGraph 초안으로 캔버스에 배치된다.
4. 사용자는 도메인별 공간, 객체, 조건, 담당자만 수정한다.

```

```

```
파일 5. `docs/03_FEATURE_REQUIREMENTS.md`
```

```
```md
```

```
# 03_FEATURE_REQUIREMENTS.md
```

```
# Feature Requirements
```

```
## 1. 기능 요구사항 개요
```

```
본 POC의 기능 요구사항은 기존 SOPBinding 관리 기능이 아니라 Visual SOP Graph Studio 구현에 초점을 둔다.
```

2. 기능 요구사항 목록

ID	기능명	설명	POC 포함
FR-001	Graph Studio Canvas	무한 캔버스 기반 노드 편집 화면	포함
FR-002	Node Palette	사용 가능한 노드 템플릿 목록 제공	포함
FR-003	Drag & Drop Node Creation	팔레트에서 캔버스로 노드 추가	포함
FR-004	Custom Node Rendering	노드 유형별 색상, 헤더, 속성, 포트 표시	포함
FR-005	Typed Input / Output Port	포트별 데이터 타입 정의	포함
FR-006	Edge Connection	노드 포트 간 연결선 생성	포함
FR-007	Connection Validation	포트 타입이 맞지 않으면 연결 제한 또는 오류 표시	포함
FR-008	Property Inspector	선택 노드의 속성 편집	포함
FR-009	Composite SOP Group	여러 SOP Task를 묶은 복합 노드	포함
FR-010	Collapse / Expand	SOP Group 접기/펼치기	포함
FR-011	Graph Validation	필수 연결, 필수 속성, 순환 참조 검증	포함
FR-012	SOPGraph JSON Export	nodes/edges를 SOPGraph로 변환	포함
FR-013	EventContext Simulator	테스트 이벤트 입력	포함
FR-014	Execution Path Highlight	실행 경로 하이라이트	포함
FR-015	Runtime Preview	임무, 전파, 응답, 상황판 반영 Mock 표시	포함
FR-016	Domain Templates	액화수소/발전소/훈련 샘플 템플릿	포함
FR-017	Real SMS	실제 문자 발송	제외
FR-018	Real IoT Integration	실제 센서/IoT 연동	제외
FR-019	3D Viewer Adapter	3D Viewer 연동 인터페이스	2차 POC
FR-020	AI/RAG SOP Generation	문서 기반 SOPGraph 초안 생성	2차 POC

3. Node Palette 요구사항

Node Palette는 다음 노드를 제공한다.

그룹	노드
Trigger	Sensor Event, Manual Report, AI Detection
Scope	Facility, Zone, Evacuation Area
Object	Sensor, Equipment, CCTV, Valve
Logic	Condition, AND/OR, Branch
SOP	SOP Task, SOP Group, Checklist
People	Role, Contact Group, Agency
Action	SMS, App Push, Broadcast
Runtime	Situation Board, Timer, Escalation
Record	Evidence, Report, History

4. Custom Node 요구사항

각 노드는 다음 요소를 포함해야 한다.

요소 설명
--- ---
Node Type event, condition, sop_group 등
Label 사용자 표시명
Description 설명
Properties 노드별 설정값
Input Ports 입력 포트
Output Ports 출력 포트
Validation State 오류/경고 상태
Collapse State 접기/펼치기 상태

5. Graph Validation 요구사항

검증 항목은 다음과 같다.

검증 항목 설명
--- ---
필수 입력 포트 required input port 연결 여부
필수 속성 eventType, condition, assignee 등 필수값
포트 타입 source/target port dataType 호환 여부
고립 노드 연결되지 않은 노드
실행 불가 경로 Event에서 Runtime까지 도달 불가능한 경로
순환 참조 무한 루프 가능성
미응답 분기 timeout 또는 escalation 누락 여부

6. Runtime Preview 요구사항

Runtime Preview는 다음 정보를 표시한다.

항목 설명
--- ---
Triggered Event 입력 EventContext
Execution Path 실행되는 노드 경로
Missions 생성된 임무
Notifications 전파 메시지 Mock
Responses 완료/지연/미응답 Mock
Situation Board Record 상황판 기록 내용

파일 6. docs/04_API_SCHEMA.md

04_API_SCHEMA.md

API Schema

1. API 개요

1차 POC는 실제 백엔드 없이 Mock API 또는 localStorage 기반으로 구현한다.
다만 향후 백엔드 전환을 고려하여 API 경로와 요청/응답 구조를 정의한다.

2. Graph API

Method	Path	설명
GET	/graphs	SOPGraph 목록 조회
GET	/graphs/:graphId	SOPGraph 상세 조회
POST	/graphs	SOPGraph 생성
PUT	/graphs/:graphId	SOPGraph 수정
DELETE	/graphs/:graphId	SOPGraph 삭제

3. Node Template API

Method	Path	설명
GET	/node-templates	사용 가능한 노드 템플릿 조회
GET	/node-templates/:type	특정 노드 타입 정의 조회

4. Validation / Compile API

Method	Path	설명
POST	/graphs/:graphId/validate	그래프 검증
POST	/graphs/:graphId/compile	ExecutionPlan 생성

5. Simulation API

Method	Path	설명
--------	------	----

```
|---|---|---|
| GET | /event-context-samples | 샘플 이벤트 목록 |
| POST | /graphs/:graphId/simulate | 이벤트 입력 후 실행 경로 계산 |
```

6. Runtime Preview API

```
| Method | Path | 설명 |
|---|---|---|
| GET | /execution-plans/:planId | 실행 계획 조회 |
| POST | /execution-plans/:planId/missions/:missionId/status | 임무 상태 Mock
변경 |
| GET | /execution-plans/:planId/timeline | 실행 타임라인 조회 |
```

7. Domain Template API

```
| Method | Path | 설명 |
|---|---|---|
| GET | /domain-templates | 도메인 템플릿 목록 |
| GET | /domain-templates/:templateId | 도메인 템플릿 상세 |
| POST | /graphs/from-template/:templateId | 템플릿 기반 SOPGraph 생성 |
```

8. Example: Create Graph

Request

```
```json
{
 "name": "가스누출 초동대응 SOPGraph",
 "domain": "plant",
 "version": "0.1.0",
 "nodes": [],
 "edges": []
}
```

## Response

```
{
 "graphId": "GRAPH-001",
 "name": "가스누출 초동대응 SOPGraph",
 "domain": "plant",
```

```
"version": "0.1.0"
}
```

---

## 9. Example: Validate Graph

### Request

```
{
 "nodes": [],
 "edges": []
}
```

### Response

```
{
 "valid": false,
 "issues": [
 {
 "level": "error",
 "nodeId": "node-sop-001",
 "message": "SOP Group 노드의 trigger 입력 포트가 연결되지 않았습니다."
 }
]
}
```

---

## 10. Example: Simulate Graph

### Request

```
{
 "eventContext": {
 "eventId": "EVT-SIM-001",
 "eventType": "GAS_LEAK",
 "severity": "WARNING",
 "source": "simulation",
 "occurredAt": "2026-06-23T10:00:00+09:00"
 }
}
```

### Response

```
{
 "planId": "PLAN-EVT-SIM-001",
 "executionPath": [
 "node-event-001",
 "node-space-001",
 "node-condition-001",
 "node-sop-group-001",
 "node-notification-001",
 "node-branch-001",
 "node-board-001"
],
 "missions": [
 {
 "missionId": "MISSION-001",
 "title": "현장 상황 확인",
 "status": "SENT"
 }
]
}
```

```

파일 7. `docs/05_SEED_DATA.md`

```md
# 05_SEED_DATA.md
# Seed Data

## 1. Seed Data 개요

1차 POC의 seed data는 특정 도메인 데이터가 아니라, 그래프 저작을 검증하기 위한 노드 템플릿
과 샘플 SOPGraph로 구성한다.

---

## 2. Node Templates

### Trigger Nodes

| Node | 설명 |
|---|---|
| Sensor Event | 센서 이벤트 트리거 |
| Manual Report | 수동 신고/제보 트리거 |
| AI Detection | AI 이상탐지 결과 트리거 |

### Scope Nodes
```

Node 설명
--- ---
Facility 시설 단위 범위
Zone 구역 단위 범위
Evacuation Area 대피구역 범위

Object Nodes

Node 설명
--- ---
Sensor 센서 객체
Equipment 설비 객체
CCTV 영상 객체
Valve 밸브 객체

Logic Nodes

Node 설명
--- ---
Condition 조건 판단
AND / OR 복합 조건
Branch 분기 처리

SOP Nodes

Node 설명
--- ---
SOP Task 단위 임무
SOP Group 복합 SOP 노드
Checklist 체크리스트

People Nodes

Node 설명
--- ---
Role 역할
Contact Group 담당자 그룹
Agency 유관기관

Action Nodes

Node 설명
--- ---
SMS 문자 전파 Mock
App Push 앱푸시 Mock
Broadcast 방송 전파 Mock

Runtime Nodes

Node	설명
---	---
Situation Board	전자상황판 기록
Timer	지연/시간 조건
Escalation	상향 전파

3. Sample Graph 1: 액화수소 플랜트

```text

가스누출 감지

- 저장구역 범위 적용
- 수소센서 객체 필터
- 위험도 WARNING 이상 조건
- 가스누출 초동대응 SOP Group
- SMS/앱 전파
- 응답 분기
- 전자상황판 기록
- 미응답 시 상향 전파

## EventContext

```
{
 "eventId": "EVT-LH2-001",
 "eventType": "GAS_LEAK",
 "severity": "WARNING",
 "source": "simulation",
 "siteId": "SITE-LH2-PLANT",
 "spaceId": "SPACE-STORAGE-ZONE",
 "assetId": "ASSET-H2-SENSOR-001",
 "measuredValues": {
 "h2_ppm": 1200
 }
}
```

## 4. Sample Graph 2: 발전소 모니터링

터빈 진동 이상

- 터빈실 공간 범위
- 터빈 설비 필터

- 진동 임계치 조건
- 터빈 점검/부하조정 SOP Group
- 책임자 보고
- 전자상황판 기록

## 5. Sample Graph 3: 안전한국훈련

- 지진 발생 훈련상황
- 건물/대피구역 범위
  - 대피유도 SOP Group
  - 교직원/담당자 전파
  - 인원확인 체크리스트
  - 상황판 기록
  - 종료보고

## 6. Sample Graph 4: 일반 사업장 안전

- 작업자 쓰러짐 신고
- 작업구역 범위
  - CCTV/현장담당자 확인
  - 응급조치 SOP Group
  - 119/관리자 전파
  - 상황판 기록

```

파일 8. `docs/06_POC_TASKS.md`

```md
# 06_POC_TASKS.md
# POC Task Breakdown

## 1. Phase 1: Project Base

| Task | 설명 |
|---|---|
| T1-01 | React + TypeScript + Vite 구성 |
| T1-02 | React Flow / xyflow 설치 |
| T1-03 | 기본 레이아웃 구성 |
| T1-04 | Graph Studio 라우트 또는 메인 화면 구성 |

```

2. Phase 2: Domain Types

Task	설명
---	---
T2-01	NodeType 정의
T2-02	PortDataType 정의
T2-03	GraphNodeData 정의
T2-04	SOPGraph 정의
T2-05	GraphEdge 정의
T2-06	EventContext 정의
T2-07	ExecutionPlan 정의

3. Phase 3: Node Templates

Task	설명
---	---
T3-01	Event Node 템플릿 작성
T3-02	Space Scope Node 템플릿 작성
T3-03	Asset Filter Node 템플릿 작성
T3-04	Condition Node 템플릿 작성
T3-05	SOP Task Node 템플릿 작성
T3-06	SOP Group Node 템플릿 작성
T3-07	Notification Node 템플릿 작성
T3-08	Branch Node 템플릿 작성
T3-09	Situation Board Node 템플릿 작성

4. Phase 4: Graph Studio UI

Task	설명
---	---
T4-01	Infinite Canvas 구현
T4-02	Node Palette 구현
T4-03	Drag & Drop Node 생성 구현
T4-04	Custom GraphNode 구현
T4-05	Input / Output Handle 표시
T4-06	Edge 연결 구현
T4-07	MiniMap / Controls / Background 적용

5. Phase 5: Property Inspector

Task	설명
T5-01	선택 노드 상태 관리
T5-02	노드 속성 표시
T5-03	JSON 기반 속성 편집
T5-04	속성 변경 시 캔버스 노드 상태 반영

6. Phase 6: Composite SOP Group

Task	설명
T6-01	SOP Group Node 렌더링
T6-02	collapsed / expanded 상태 관리
T6-03	내부 Task 목록 표시
T6-04	접기/펼치기 UI 구현
T6-05	children node 연결 구조 반영

7. Phase 7: Graph Validation

Task	설명
T7-01	필수 입력 포트 검증
T7-02	필수 속성 검증
T7-03	포트 타입 검증
T7-04	고립 노드 검증
T7-05	Event에서 Runtime까지 도달 가능성 검증
T7-06	Validation Panel 표시

8. Phase 8: Graph Compile

Task	설명
T8-01	React Flow nodes/edges normalize
T8-02	SOPGraph JSON 생성
T8-03	EdgeType 매핑
T8-04	Compile 결과 Preview
T8-05	localStorage 저장

9. Phase 9: Event Simulation

Task	설명
T9-01	샘플 EventContext 작성
T9-02	Trigger Node 매칭
T9-03	Condition Node 평가 Mock
T9-04	실행 경로 계산
T9-05	실행 경로 하이라이트

10. Phase 10: Runtime Preview

Task	설명
T10-01	RuntimeMission 생성
T10-02	Notification Mock 생성
T10-03	Branch 결과 Mock 표시
T10-04	Situation Board 기록 Mock 표시
T10-05	Execution Timeline 표시

11. Phase 11: Domain Templates

Task	설명
T11-01	액화수소 플랜트 샘플 그래프
T11-02	발전소 모니터링 샘플 그래프
T11-03	안전한국훈련 샘플 그래프
T11-04	일반 사업장 안전 샘플 그래프

12. Phase 12: Review

Task	설명
T12-01	Figma v2 와이어프레임과 UI 비교
T12-02	Graph Studio 사용성 검토
T12-03	SOPGraph JSON 구조 검토
T12-04	2차 POC 범위 정리